Desarrollo de Juegos

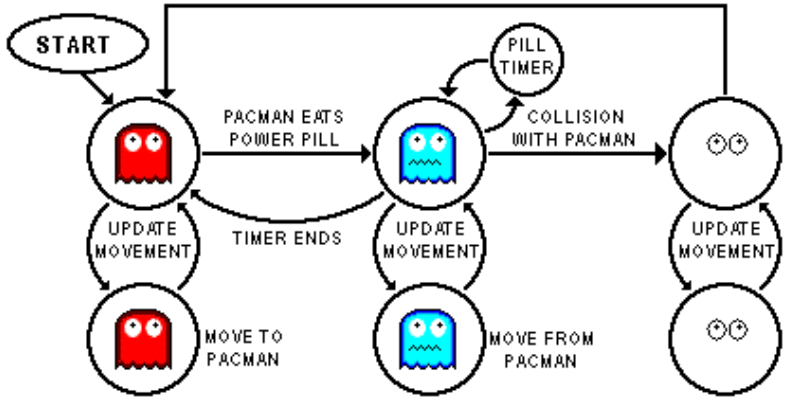
Unidad de Trabajo 5

Contenidos

1. Diseño y definición de la lógica del juego
2. Introducción a desarrollo de juegos con LibGDX
3. Creación de objetos del juego (Game Objects)
4. Escenas, pantallas y distribución de elementos
5. Gráficos, materiales y texturas
6. Física y comportamiento de los objetos
7. Sonido y efectos audiovisuales
8. Cámaras e iluminación
9. Desarrollo multiplataforma: Desktop + Android

Diseño y definición de la lógica del juego

- **Diseño**
  - Concepto de gameplay y mecánicas de juego.
  - Definición de reglas, objetivos y estados del juego.
  - Modelado de la lógica del juego:
    - Bucles de juego.
    - Gestión de estados (menús, juego, pausa, fin).
  - Diagramas sencillos de estados y flujos de juego.



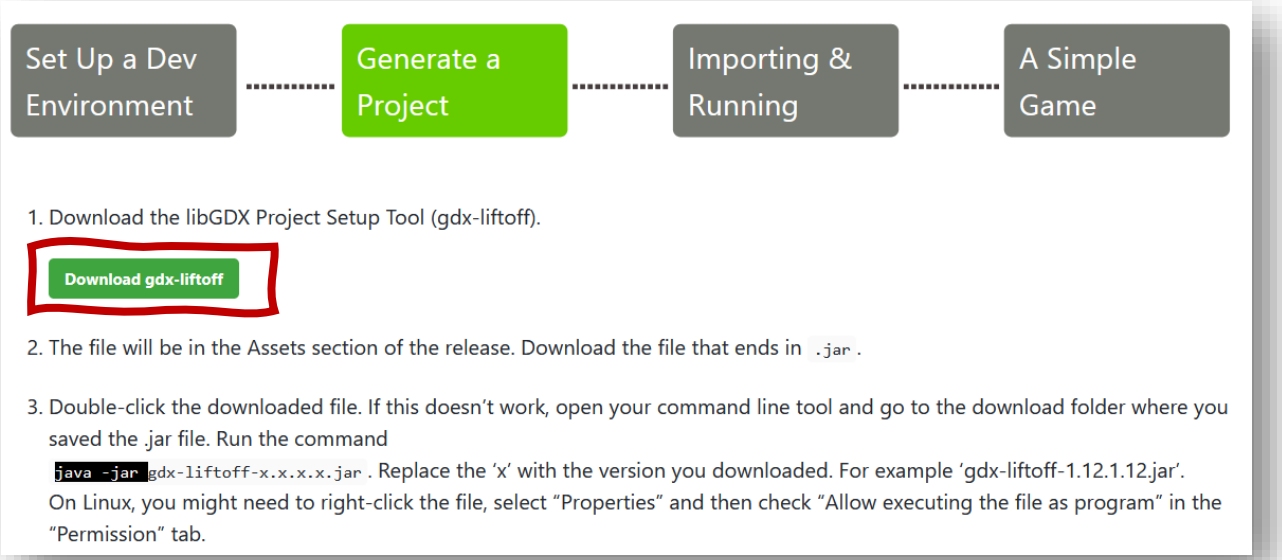
Juego propuesto: *"Space Escape"*
Nave que esquiva/dispara a obstáculos y suma puntos
2D sencillo multiplataforma usando LibGDX
Incorporando lógica de juego, objetos, escenas, física básica, sonido y despliegue en móvil

Diseño y definición de la lógica del juego

- **Descripción del juego final**
  - El jugador controla una **nave espacial**.
  - Aparecen **obstáculos** que se mueven hacia el jugador.
  - El jugador gana **puntos por tiempo sobrevivido y por destruir obstáculos**.
  - El juego termina al colisionar con un obstáculo.
  - Incluye:
    - Menú inicial
    - Pantalla de juego
    - Pantalla Game Over
    - Sonidos y música
    - Control por teclado (desktop) y táctil (móvil)

Introducción a desarrollo de juegos con LibGDX

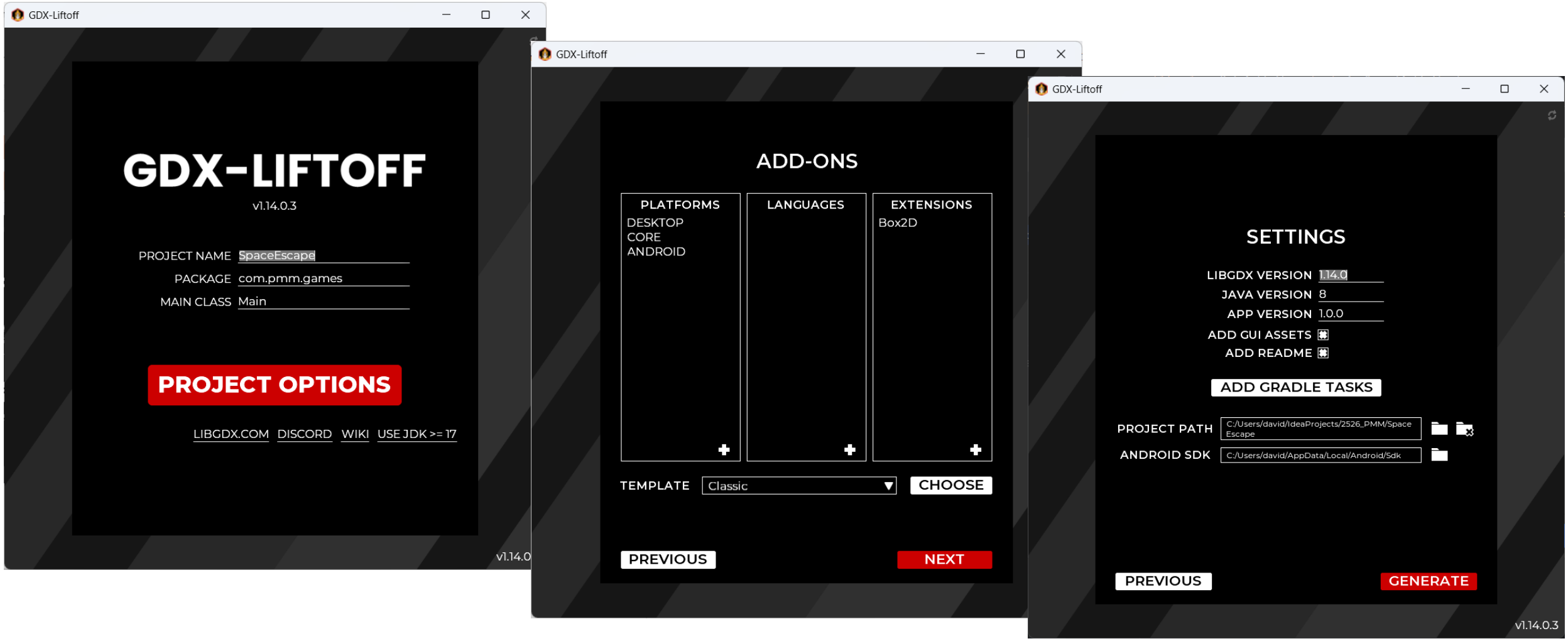
- Creación del proyecto:
  - Uso del LibGDX Project Generator.
  - Selección de:
    - Módulos: core, desktop, android.
    - Extensiones: Box2D (para la física)



  - Los proyectos de libGDX se manejan con Gradle, como en AndroidStudio (p.e.: dependencias, build…).

Introducción a desarrollo de juegos con LibGDX

- **Creación del proyecto**



Introducción a desarrollo de juegos con LibGDX

- **Estructura del proyecto**

```
gradle.properties          <- global variables used to define version numbers throughout the project
settings.gradle            <- definition of sub-modules. By default core, desktop, android, html, ios
build.gradle               <- main Gradle build file
gradlew                    <- local Gradle wrapper
gradlew.bat                <- script that will run Gradle on Windows
local.properties           <- IntelliJ only file, defines Android SDK location

assets/                    <- contains your graphics, audio, etc.

core/
    build.gradle           <- Gradle build file for core project. Defines dependencies throughout the project.
    src/                   <- Source folder for all your game's code

lwjgl3/
    build.gradle           <- Gradle build file for desktop project. Defines desktop only dependencies.
    src/                   <- Source folder for your desktop project, contains LWJGL launcher class

android/
    build.gradle           <- Gradle build file for android project. Defines Android only dependencies.
    AndroidManifest.xml    <- Android specific config
    res/                   <- contains icons for your app and other resources
    src/                   <- Source folder for your Android project, contains android launcher class
```

***LWJGL** = [Lightweight Java Game Library]*

Introducción a desarrollo de juegos con LibGDX

- **Estructura del proyecto**
  - Es necesario pasar, en el build.gradle de la versión 8.9.3 de Android Gradle Plugin (AGP) que viene con LibGDX a la 8.6.0-alpha07 que es con la que trabaja Intellij.

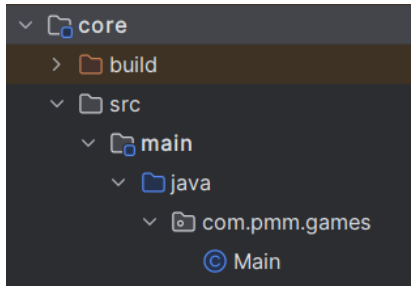
```
build.gradle (SpaceEscape)
1   buildscript {
9     dependencies {
10      classpath "com.android.tools.build:gradle:8.9.3"
```

```
build.gradle (SpaceEscape)
1   buildscript {
9     dependencies {
10      //noinspection AndroidGradlePluginVersion
11      classpath "com.android.tools.build:gradle:8.6.0-alpha07"
```

  - Después debemos recargar Gradle y al compilar no debería dar problemas.

Primer juego funcional

- Implementación en Core de:
  - Clase principal (Game)




```java
public class Main extends Game {
    private SpriteBatch batch;
    private Texture image;
    private BitmapFont font;

    @Override
    public void create() {
        batch = new SpriteBatch();
        image = new Texture("libgdx.png");
        font = new BitmapFont();
    }
```

  - Método render() → ejecutado ~ 60 veces/s
    - Dibujar un fondo simple con SpriteBatch.
    - Cargar una imagen de fondo.
    - Mostrar texto: “Space Escape”.

```java
@Override
public void render() {
    ScreenUtils.clear(0.15f, 0.15f, 0.2f, 1f);

    batch.begin();

    batch.draw(image, 140, 210);
    font.draw(batch, "Space Escape", 100, 150);
    font.draw(batch, "Pulsa una tecla para empezar", 100, 100);

    batch.end();
}
```

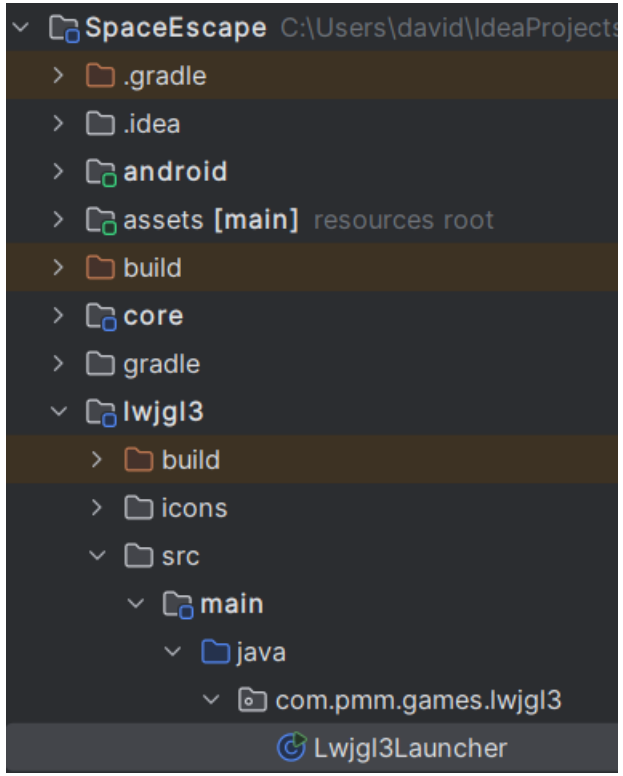
  - Método dispose()

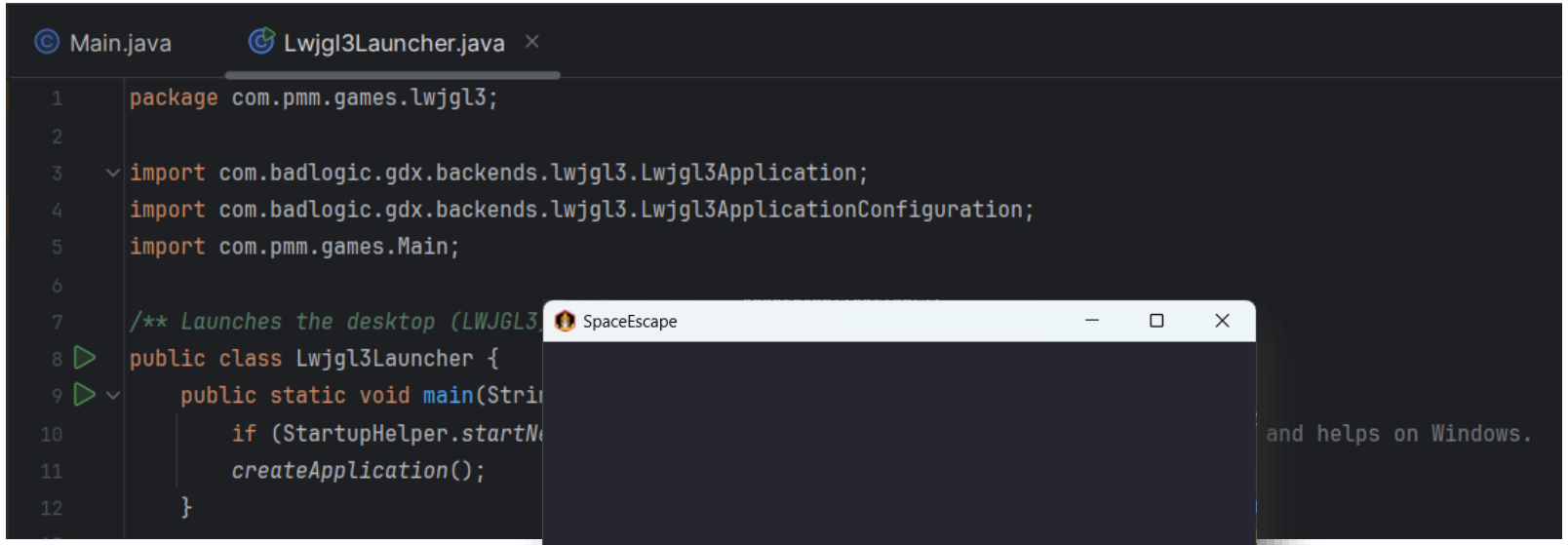
```java
@Override
public void dispose() {
    batch.dispose();
    image.dispose();
    font.dispose();
}
```

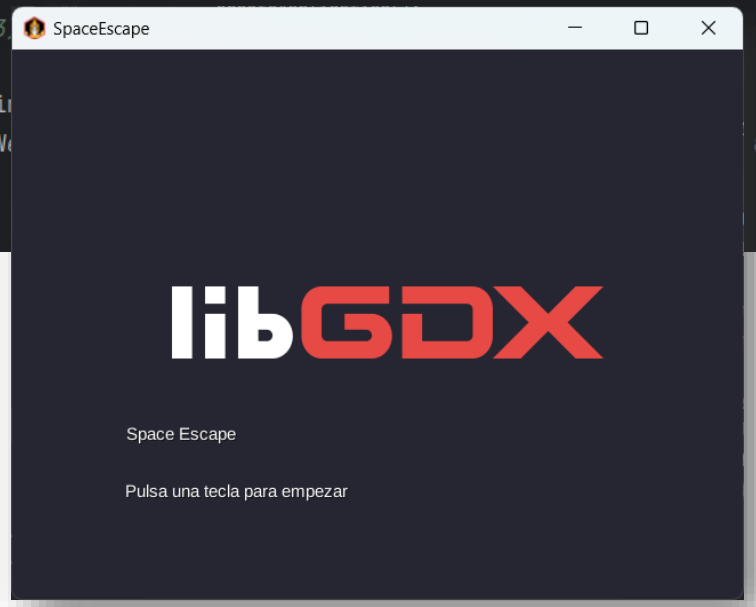
Un juego LibGDX es un **programa que se ejecuta muchas veces por segundo**, dibujando y actualizando cosas en pantalla.

Primer juego funcional

- Ejecución **en ordenador**:
  - Mediante la clase **Lwjgl3Launcher**.

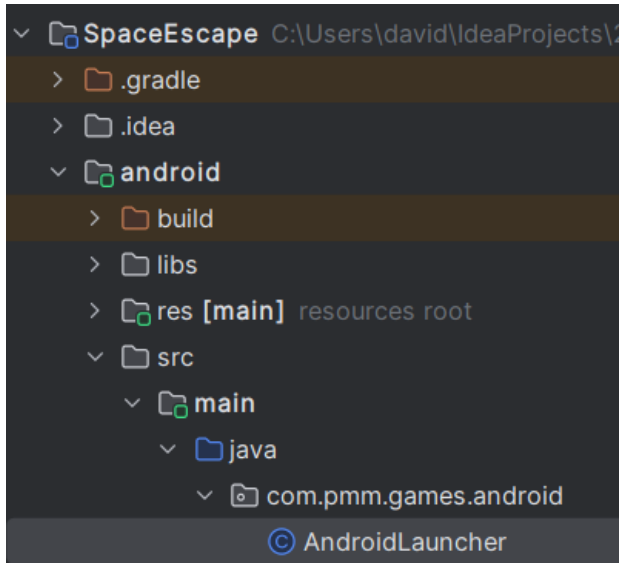


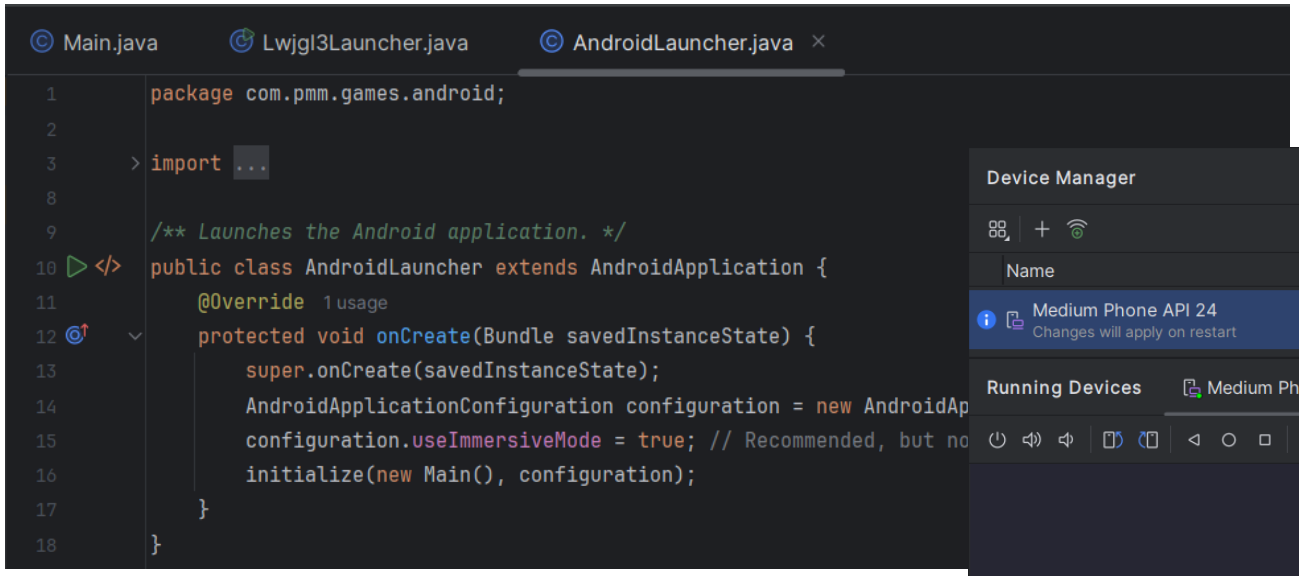


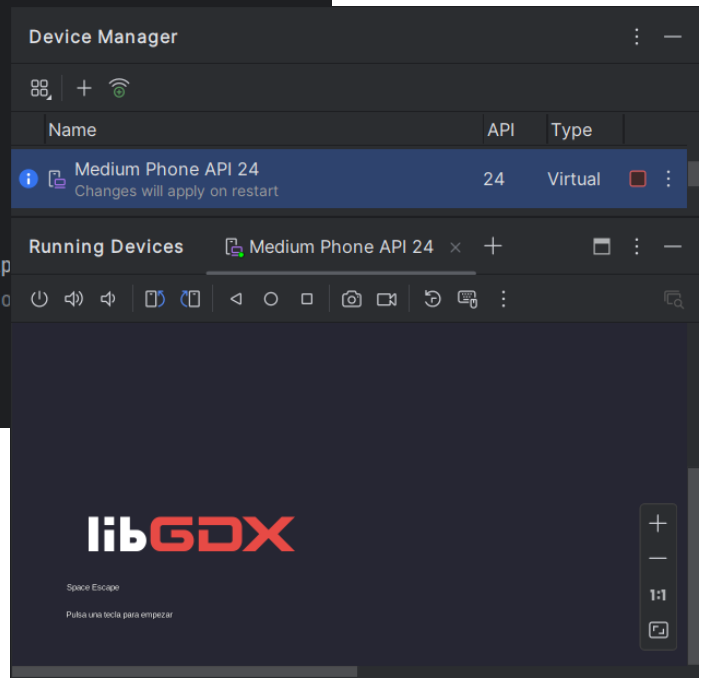


Primer juego funcional

- Ejecución **en dispositivo Android**:
  - Mediante la clase **AndroidLauncher**.







Los AVD pertenecen al sistema, no al IDE. Por tanto, **los emuladores creados en Android Studio son detectados automáticamente por IntelliJ**.

Primer juego funcional

- Definir estados:
  - MENÚ
  - JUGANDO
  - GAME OVER

```java
public enum GameState {
    MENU,  4 usages
    PLAYING,  3 usages
    GAME_OVER  3 usages
}
```

- Control de flujo del juego con variable de estado.

```java
private void representacionEstado() {  1 usage
    switch (gameState) {

        case MENU:
            // ESCENA: Render escena
            int screenWidth = Gdx.graphics.getWidth();
            int screenHeight = Gdx.graphics.getHeight();
            batch.draw(gameLogo,  x: 0,  y: 0, screenWidth, screenHeight);
            //font.draw(batch, "SPACE ESCAPE", 100, 150);
            font.draw(batch,  str: "Pulsa ESPACIO para jugar",  x: 100,  y: 100);
            break;
```

- Cambio de estado al pulsar una tecla:
  - El juego no cambia de pantalla.
  - Cambia su estado interno y con él las acciones disponibles.

```java
private void gestionarInputs() {  1 usage

    switch (gameState) {

        case MENU:
            // ESTADOS: Cambio de estado
            if (Gdx.input.isKeyJustPressed(Input.Keys.SPACE)) {
                nextGameState = GameState.PLAYING;
                gameStateChanged= true;
            }
            break;
```

Un estado del juego indica *qué está pasando ahora mismo* y determina **qué se puede hacer y qué se dibuja.**

Practicando

- **PROBAD A:**

1. Cambiar el color del fondo según el estado del juego.

2. Introduce una nueva imagen en el estado GAME_OVER (ruta: */assets*)

Objetos del juego

- **Creación del jugador**
  - Clase Player
  - Propiedades:
    - Posición
    - Tamaño
    - Velocidad
  - Representación en pantalla (resolución / límites)
  - Movimiento (teclado / pulsaciones)

En LibGDX, un jugador es normalmente un **sprite o una textura** que se dibuja con `SpriteBatch` y que tiene **atributos propios**:

- `x` e `y` → posición en pantalla
- `width` y `height` → tamaño del jugador
- `speed` → velocidad de movimiento
- `texture` → imagen que representa al jugador

Se puede implementar como clase propia para tener **lógica encapsulada**.

Objetos del juego

- **Obstáculos del juego**
  - Clase Obstacle.
  - Requiere:
    - Generación automática de obstáculos/enemigos.
    - Movimiento vertical hacia el jugador.
    - Detección de colisiones con el jugador
    - Eliminación de obstáculos que salen de la pantalla
  - Listas de objetos en movimiento

Un obstáculo es técnicamente **muy similar al jugador**, pero:

- No responde a la entrada del usuario
- Se actualiza siempre en update()

Objetos del juego

- **Colisiones**
  - Añadir *bounds* al jugador y a los obstáculos:
    - La manera más simple es mediante un rectángulo.
    - Deben desplazarse junto al objeto al que se asocian.
    - Definen una zona “lógica” de colisión.
  - Comprobación de colisiones:
    - En cada frame.
    - Analizamos si el *bound* del player se solapa - *rectangle.overlaps(...)* - con el de alguno de los obstáculos.

- Colisiones **manuales**, rápidas y sencillas
- La colisión **no tiene por qué coincidir con la imagen** - coincide con lo que queremos que el jugador sienta como impacto.

Objetos del juego

- **Colisiones**
  - Integración del motor de física (Box2D) en LibGDX:
    - Detecta colisiones
    - Aplica fuerzas, velocidades, masas…
    - Gestiona rebotes, gravedad, fricción.

  - Requiere crear:
    - Un mundo físico y establecer su escala.
    - El cuerpo físico del jugador, con su forma y propiedades.
    - Establecer el movimiento con mecanismos físicos (velocidades, fuerzas…).
    - Sincronizar gráfico con física.
    - Definir *listeners* para las colisiones.

Con un motor de física no movemos objetos
Movemos cuerpos y el motor hace el resto.

Conceptos relevantes

- Encapsulación (Player)
- Entrada multiplataforma (isKeyPressed + isTouched)
- Adaptación a resolución
- Control de límites
- Separación lógica vs renderizado

- … y no olvidéis hacer dispose() de todos los elementos.

Escenas

- **Gestión de escenas**
  - Hemos utilizado la aproximación básica para las pantallas, pero podemos recurrir al uso de escenas (*Screen*).
  - Permite separar la lógica y el render de cada escena.
  - Se integran en la librería Scene2D.
  - Nos facilita el uso de skins. Agregar al menos las siguientes a */assets*:
    - uiskin.json
    - uiskin.atlas
    - uiskin.png
    - default.fnt
    - default.png
  - Debemos gestionar el cambio entre pantallas.

Practicando

- **PROBAD A:**

1. Añadir marcador de puntos (contador de tiempo) que se vean reflejados en la ScoreScreen.

2. Añadir disparos de la nave cuando el usuario pulsa una tecla y que destruyan los asteroides al colisionar con ellos.

3. Sumar puntos por la destrucción de asteroides.

Sonido y efectos audiovisuales

- Tipos de sonido en videojuegos:
  - Efectos (SFX) → Sound
  - Música de fondo → Music
- Uso de la API de audio de LibGDX:
  - Sound
  - Music
- Asociación de sonidos a eventos:
  - Acciones del jugador.
  - Colisiones.
  - Cambios de estado.
- Gestión de volumen y recursos de audio.

Un juego sin audio **funciona**, pero se percibe:

- Menos inmersivo
- Más aburrido
- Con feedback insuficiente
- Menos profesional
- Difícil de interpretar en situaciones críticas

Cámaras e iluminación

- En LibGDX, incluso en 2D, **todo se dibuja a través de una cámara**.
- La cámara controla:
  - Qué parte del mundo se ve
  - El escalado en distintas resoluciones
  - El desplazamiento (scroll)

Coordenadas = mundo lógico del juego
Pantalla = solo una ventana a ese mundo

- En Space Escape:
  - Estamos utilizamos una OrthographicCamera, adecuada para juegos 2D.
  - Se genera implícitamente, pero no podemos controlarla (fija, sin scroll ni zoom, sin cambio de resolución...).
- Para tener control sobre la cámara:

Cámaras e iluminación

- Para tener control sobre la cámara deberemos crearle en *create()*:

```
camera = new OrthographicCamera();
// Resolución lógica del juego
viewport = new ExtendViewport(WORLD_WIDTH, WORLD_HEIGHT,
camera); // FitViewport
viewport.apply();
camera.position.set(WORLD_WIDTH / 2, WORLD_HEIGHT / 2, 0);
camera.update();
```

- Y necesitaremos sobrescribir el método *show()*:

```
@Override
public void resize(int width, int height) {
    viewport.update(width, height, true);
}
```

Cámaras e iluminación

- LibGDX simula la iluminación mediante:
  - Colores
  - Transparencias
  - Superposiciones →
  - Efectos visuales

```
// ILUMINACIÓN: Explosión
if (flashActive) {
    batch.setColor(1f, 1f, 1f, 0.6f); // más visible
    batch.draw(whitePixel, 0, 0, screenWidth, screenHeight);
    batch.setColor(1f, 1f, 1f, 1f); // restablecer
}
```

- También los shaders permiten controlar color, brillo y efectos de forma mucho más precisa **(es un efecto de postprocesado)**
  - Vertex shader → transforma vértices y pasa color + coordenadas a fragment shader
  - Fragment shader → calcula color final de cada pixel. Podemos pasarle un modificador para hacer un destello de luz instantáneo.

Cámaras e iluminación

- Para los shaders es necesario:
  - Generar los ficheros .vert y .frag
  - Cargar los dos ficheros
  - Generar ShaderProgram con ellos
  - Compilar el programa

```
// SHADER: cargar shader
String vertex = Gdx.files.internal("shaders/shoot.vert").readString(); // SHADER
String fragment = Gdx.files.internal("shaders/shoot.frag").readString(); // SHADER
shootShader = new ShaderProgram(vertex, fragment); // SHADER
if (!shootShader.isCompiled()) { // SHADER
    Gdx.app.error("Shader", shootShader.getLog()); // SHADER
}
```

  - Aplicarlo en el renderizado cuando sea necesario

```
// SHADER: aplicar shader
batch.setShader(shootShader); // SHADER
shootShader.setUniformf("u_flash", shootIntensity); // SHADER
batch.begin();
....
batch.end();
batch.setShader(null);
```

Practicando

- **PROBAD A:**

1. Añadir vuestra propia música y efectos de sonido.
2. Introducid alguna adaptación al juego para hacerlo vuestro.

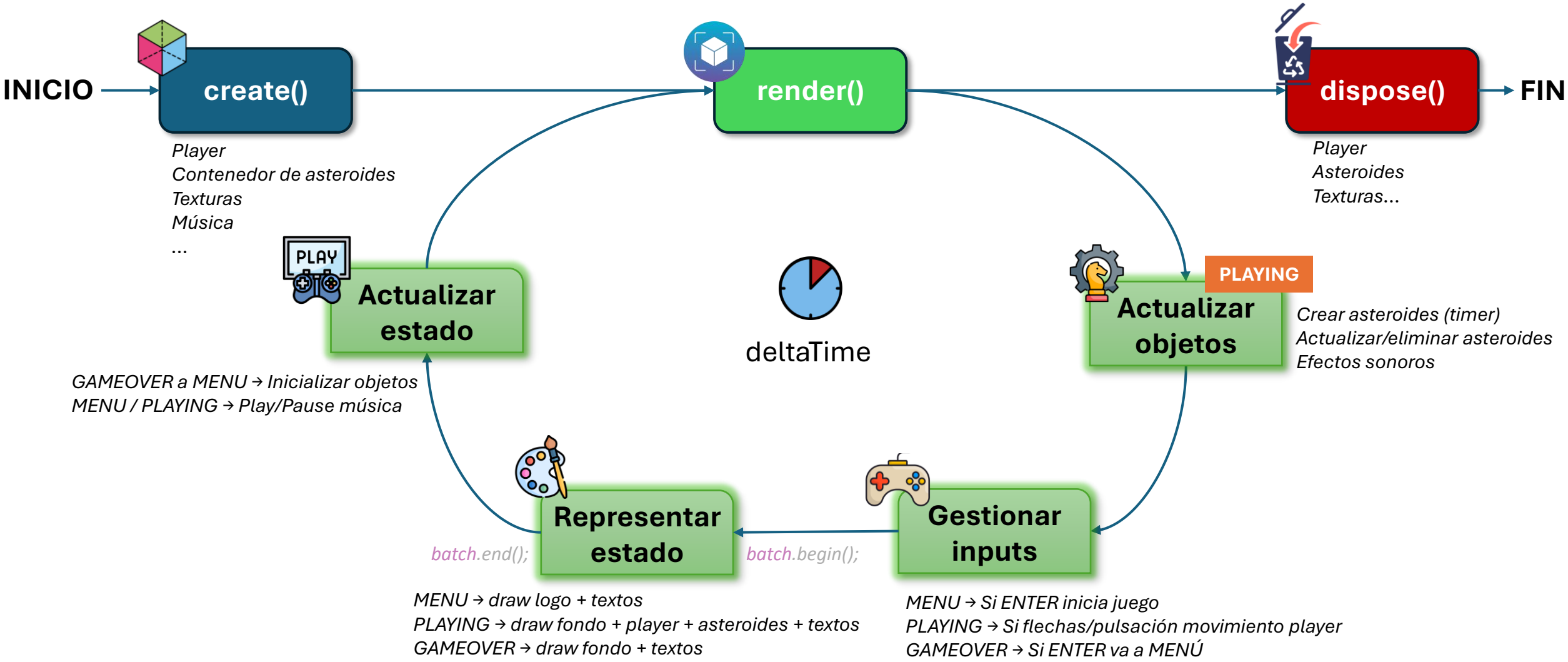



Diagrama de flujo del juego